

ARGOS — Documentação Técnica

Sistema de Web

Stack Resumida — O que usamos no Frontend e na API

Camada	Tecnologia Principal	Linguagem
Frontend (Interface)	React 19 via Next.js 16	TypeScript 5.7
Backend (API)	Next.js 16 API Routes (Node.js)	TypeScript 5.7
Banco de Dados	Cloud Firestore (Google Firebase)	—
Autenticação	Firebase Auth	—
Estilização	Tailwind CSS 4 + shadcn/ui	CSS (utility-first)

Frontend — React com Next.js

O frontend é construído com **React 19**, a biblioteca JavaScript mais usada no mundo para criar interfaces de utilizador. O React permite construir a UI como uma árvore de **componentes reutilizáveis** — cada card do Kanban, cada tabela, cada formulário é um componente independente.

O **Next.js 16** é o framework que envolve o React e adiciona: roteamento automático por pastas, renderização server-side (SSR), otimizações de performance e — o mais importante para nós — as **API Routes**, que permitem criar o backend no mesmo projeto.

A linguagem usada em todo o projeto é **TypeScript**, que é JavaScript com tipagem estática. Isso significa que o editor avisa erros de tipo antes mesmo de executar o código.

API (Backend) — Node.js com TypeScript

A API é construída com os **Route Handlers do Next.js**, que correm sobre o **Node.js** (runtime JavaScript para servidor). Não usamos Express, Fastify ou nenhum outro framework de servidor separado — o próprio Next.js gere as rotas HTTP.

Cada ficheiro **route.ts** dentro de **app/api/** é um endpoint. Dentro desses ficheiros, usamos o **Firebase Admin SDK** (biblioteca oficial do Google) para ler e escrever no Firestore com permissões de administrador.

```
app/api/sinistros/route.ts → GET/POST /api/sinistros
app/api/sinistros/[id]/route.ts → GET/PATCH/DELETE /api/sinistros/{id}
```

Sumário

- 1. Estrutura de Pastas e Padrão Arquitetural

- 2. Por Que Não Tudo Num Único Ficheiro?
- 3. Fluxo de Integração Frontend ↔ Backend
- 4. Como Funciona Cada API
- 5. Segurança das APIs
- 6. Mapeamento de Rotas por Componente
- 7. Tecnologias e Justificativas
- 8. Comunicação Frontend ↔ API (HTTP/REST)
- 9. Mecanismos de Segurança Implementados
- 10. Tratamento de Formulários Inválidos
- 11. Simulação de Banca — 5 Perguntas Difíceis
- 12. Atualização em Tempo Real e Integrações Externas

1. Estrutura de Pastas e Padrão Arquitetural

Estrutura de Diretórios

```
web-argos/
├── app/                                # Roteamento e páginas (Next.js App Router)
│   ├── api/                            # Backend – endpoints REST (API Routes)
│   │   ├── dashboard/                 # Agregação de KPIs e dados do painel
│   │   ├── sinistros/                # CRUD + lógica do Kanban de sinistros
│   │   ├── vistorias/                # Gestão de vistorias periciais
│   │   ├── clientes/                 # Registo de clientes
│   │   ├── veiculos/                 # Registo de veículos
│   │   ├── oficinas/                 # Rede credenciada (oficinas)
│   │   ├── seguradoras/              # Seguradoras
│   │   ├── parceiros/                # Fluxo de aprovação de parceiros
│   │   └── alertas/                  # Central de alertas
│   ├── dashboard/                     # Página do painel principal
│   ├── orquestracao/                  # Quadro Kanban de sinistros
│   ├── registros/                     # Páginas de registo (clientes, veículos, users)
│   ├── rede-credenciada/              # Gestão de oficinas credenciadas
│   ├── alertas/                       # Central de alertas
│   ├── vistoria/                      # Detalhes de uma vistoria
│   ├── login/                         # Autenticação
│   └── logout/                        # Encerramento de sessão
├── components/                         # Componentes React reutilizáveis
│   ├── dashboard/                     # Cards de KPI, gráficos, tabelas
│   ├── orquestracao/                  # Kanban Board com drag-and-drop
│   ├── alertas/                       # UI de alertas
│   ├── credenciados/                  # Lista de oficinas, gráfico de qualidade
│   ├── registros/                     # Formulários e tabelas de registo
│   ├── layout/                        # AppLayout, Sidebar, Header, LoadingScreen
│   └── ui/                            # ~40 componentes base (shadcn/ui)
├── lib/                               # Utilitários e serviços partilhados
│   ├── firebase.ts                    # Inicialização Firebase (lado do cliente)
│   └── firebase-admin.ts              # Inicialização Firebase Admin (lado do
servidor)
```

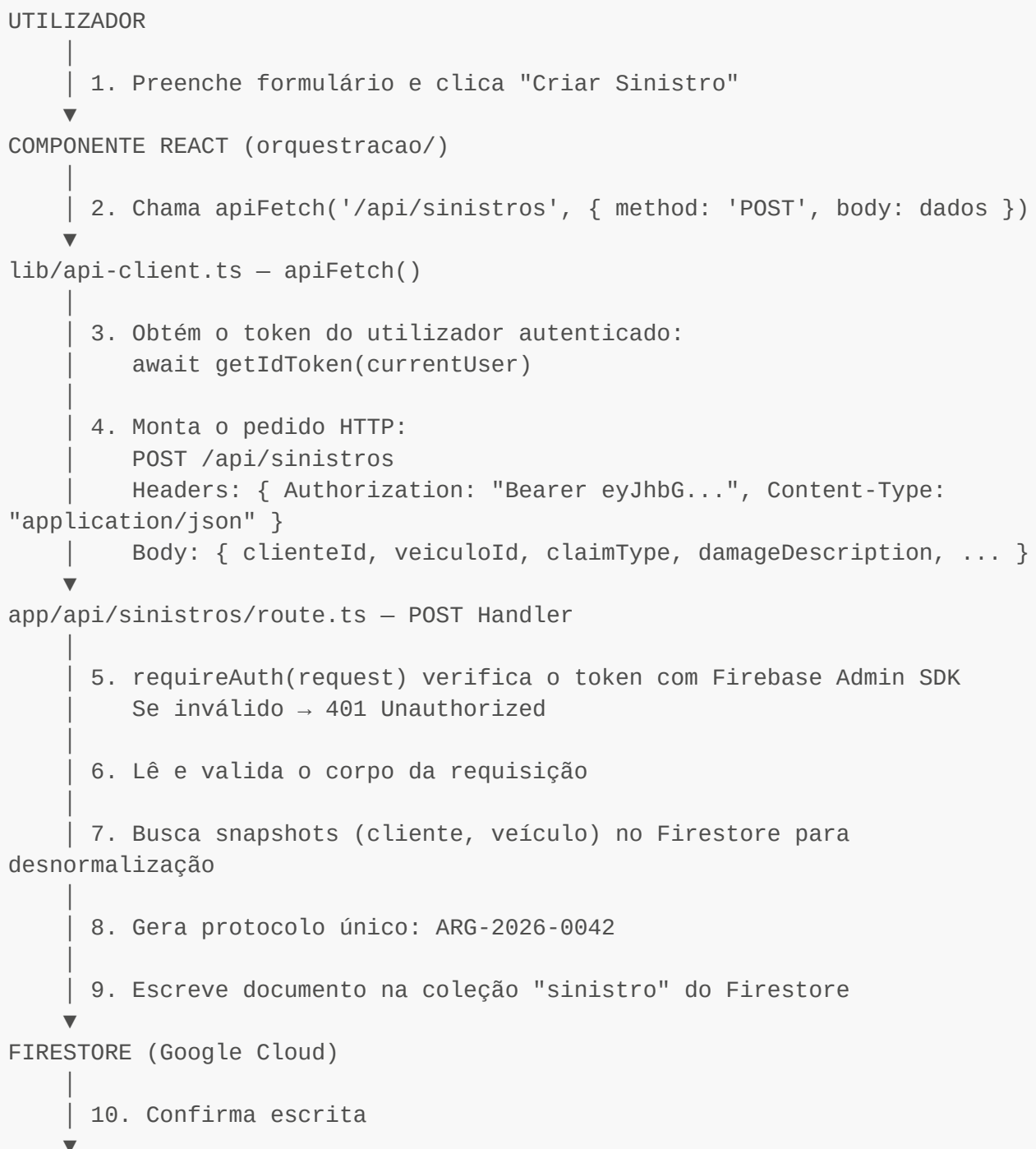

3. Testabilidade Funções isoladas em `lib/services/auth.ts` ou `lib/auth-server.ts` podem ser testadas unitariamente sem instanciar um servidor HTTP inteiro.

4. Reutilização O `apiFetch()` em `lib/api-client.ts` é usado por todos os componentes do sistema. Sem separação, cada componente repetiria o código de injeção de token — violando o princípio DRY (Don't Repeat Yourself).

5. Trabalho em Equipe Separação de ficheiros evita conflitos de merge no Git. Dois desenvolvedores podem trabalhar simultaneamente em `clientes/route.ts` e `veiculos/route.ts` sem colisão.

3. Fluxo de Integração Frontend ↔ Backend

Exemplo: Criar um Sinistro



```
app/api/sinistros/route.ts
|
| 11. Devolve resposta:
|     HTTP 201 Created
|     { success: true, id: "abc123", protocol: "ARG-2026-0042" }
|
▼
COMPONENTE REACT
|
| 12. Exibe toast de sucesso
| 13. Revalida os dados do SWR (mutate())
| 14. Fecha o modal / redireciona
```

4. Como Funciona Cada API

`/api/dashboard` — Painel Central

Agrega dados de toda a base em memória. Lê todos os sinistros e vistorias, classifica cada sinistro nas 5 colunas do pipeline (aguardando vínculo, check-in, em vistoria, etc.) e calcula KPIs. Devolve também o gráfico de severidade agrupado por data e os alertas mais recentes.

`/api/sinistros` — Gestão de Sinistros

CRUD completo com lógica de Kanban. Suporta múltiplas views (geral, rejeitadas, alertas IA, kanban). Ao criar, gera protocolo automático `ARG-YYYY-####`. Ao mover de estágio (`/stage`), atualiza o status e registra timestamps. O endpoint `/vincular` atribui uma oficina ao sinistro (`credenciadoSnapshot`).

`/api/vistorias` — Vistorias Periciais

Gere o ciclo de vida das vistorias: criação, agendamento, execução, análise e finalização. Armazena laudos, imagens em base64, áudios com transcrições e histórico de chat. Gera IDs sequenciais via coleção `counters` (padrão de contador distribuído do Firestore).

`/api/clientes` e `/api/veiculos` — Registos

CRUD padrão com busca por CPF/CNPJ, placa ou nome. Retornam listas paginadas.

`/api/oficinas` — Rede Credenciada

Listagem com filtro em memória (evita índices compostos no Firestore). Suporta busca por prefixo de nome e filtragem por especialidade/cidade/UF.

`/api/alertas` — Central de Alertas

Lê a coleção `alertas` com filtros por tipo (`critico`, `sla`, `info`) e por status de leitura. Suporta marcar alertas como lidos.

`/api/parceiros/aprovar-email` — Aprovação de Parceiros

Gera um link de redefinição de senha via Firebase Admin Auth (`generatePasswordResetLink`) e o devolve ao frontend para envio. O e-mail é disparado pela infraestrutura do próprio Firebase.

5. Segurança das APIs

Todas as rotas exigem autenticação. O mecanismo funciona em três camadas:

Camada 1 — Firebase Authentication (Frontend)

O utilizador faz login com e-mail/password ou Google OAuth. O Firebase devolve um **ID Token JWT** assinado pelos servidores do Google, com validade de 1 hora.

Camada 2 — Middleware `requireAuth()` (Backend)

Cada route handler começa com:

```
const user = await requireAuth(request);  
// Se falhar → lança erro 401 automaticamente
```

O `requireAuth()` em `lib/auth-server.ts`:

1. Extrai o header `Authorization: Bearer <token>`
2. Chama `getAdminAuth().verifyIdToken(token)` — valida a assinatura criptográfica do JWT contra os certificados públicos do Google
3. Se o token for inválido, expirado ou falsificado → devolve `401 Unauthorized`
4. Se válido → devolve os dados do utilizador autenticado

Camada 3 — Separação de Credenciais

As chaves privadas do Firebase Admin (`FIREBASE_PRIVATE_KEY`, `FIREBASE_CLIENT_EMAIL`) existem **apenas no servidor** como variáveis de ambiente. O Next.js garante que variáveis sem o prefixo `NEXT_PUBLIC_` nunca chegam ao browser.

O que está protegido

- Nenhum endpoint da API responde sem token válido
- O Firestore é acedido pelo servidor (Admin SDK), nunca diretamente pelo browser
- Dados sensíveis (chaves de serviço) nunca chegam ao cliente

6. Mapeamento de Rotas por Componente

Componente	Rota Consumida	Operação
<code>dashboard/kpi-cards.tsx</code>	<code>GET /api/dashboard</code>	Carrega KPIs e pipeline
<code>dashboard/severity-chart.tsx</code>	<code>GET /api/dashboard</code>	Dados do gráfico diário

Componente	Rota Consumida	Operação
dashboard/recent-claims-table.tsx	GET /api/dashboard	Lista paginada de sinistros
dashboard/alerts-feed.tsx	GET /api/dashboard	Últimos 4 alertas
orquestracao/kanban-board.tsx	GET /api/sinistros?view=kanban	Colunas do Kanban
orquestracao/kanban-board.tsx	PATCH /api/sinistros/{id}/stage	Mover card de coluna
orquestracao/kanban-board.tsx	POST /api/sinistros/{id}/vincular	Atribuir oficina
orquestracao/kanban-board.tsx	POST /api/sinistros/{id}/checkin	Registrar check-in
credenciados/oficinas-list.tsx	GET /api/oficinas	Lista de oficinas
registros/(clientes)	GET/POST /api/clientes	Listar e criar clientes
registros/(veiculos)	GET/POST /api/veiculos	Listar e criar veículos
alertas/alertas-feed.tsx	GET /api/alertas	Lista de alertas
app/vistoria/[id]	GET /api/vistorias/{id}	Detalhes da vistoria
app/login/page.tsx	Firebase Auth SDK (direto)	Login / Google OAuth

7. Tecnologias e Justificativas

Tecnologia	Justificativa Técnica
Next.js 16 (App Router)	Unifica frontend e backend num único projeto, elimina a necessidade de um servidor Express separado e oferece SSR/SSG nativos para performance.
TypeScript 5.7	Tipagem estática previne erros em tempo de compilação, essencial num sistema com múltiplas entidades de dados (sinistros, vistorias, clientes) que interagem entre si.
Firebase Auth	Abstrai toda a complexidade de gestão de sessões, tokens JWT e OAuth2, oferecendo autenticação production-ready sem implementar criptografia manualmente.
Firestore (NoSQL)	Modelo de documentos aninhados é ideal para snapshots desnormalizados (cliente/veículo dentro do sinistro), evitando JOINS custosos num sistema de leitura intensiva.
Firebase Admin SDK	Permite verificar tokens e aceder ao Firestore com permissões de administrador exclusivamente no servidor, sem expor credenciais ao frontend.
React Hook Form + Zod	A combinação oferece validação declarativa com tipagem TypeScript automática a partir do schema, reduzindo código boilerplate nos formulários.

Tecnologia	Justificativa Técnica
SWR (Stale-While-Revalidate)	Estratégia de cache que mostra dados imediatos enquanto revalida em background, tornando a navegação instantânea sem sacrificar a consistência dos dados.
Tailwind CSS 4	Utility-first elimina a necessidade de ficheiros CSS separados e nomeação de classes, acelerando o desenvolvimento de UI e garantindo consistência visual.
shadcn/ui + Radix UI	Componentes acessíveis (WCAG) com comportamento correto para teclado e screen readers, sem bloquear a personalização visual do produto.
Recharts	Biblioteca de gráficos construída sobre D3.js mas com API declarativa React, permitindo o gráfico de severidade com menos de 50 linhas de código.
pnpm	Gestor de pacotes com hard links que reduz o espaço em disco em ~50% e acelera installs, especialmente em pipelines CI/CD.
Firebase App Hosting	Deploy contínuo com suporte nativo a Next.js, gestão de secrets integrada e região southamerica-east1 para menor latência para utilizadores brasileiros.

8. Comunicação Frontend ↔ API (HTTP/REST)

Protocolo e Padrão

Utilizamos **HTTP/REST** sobre HTTPS. Cada recurso tem uma URL semântica e os verbos HTTP expressam a intenção: **GET** para leitura, **POST** para criação, **PUT/PATCH** para atualização, **DELETE** para remoção.

O Wrapper `apiFetch()` — `lib/api-client.ts`

Todos os pedidos passam por este wrapper, que:

1. Aguarda o Firebase estar pronto (`onAuthStateChanged`)
2. Obtém o token do utilizador atual (`getIdToken(currentUser)`)
3. Monta os headers obrigatórios
4. Executa o `fetch()` nativo do browser

Pedido típico enviado:

```
POST /api/sinistros HTTP/1.1
```

```
Host: argos.app
```

```
Authorization: Bearer eyJhbGciOiJSUzI1NiIsImtpZCI6... ← token JWT do  
Firebase
```

```
Content-Type: application/json ← formato do corpo
```

```
{  
  "clienteId": "123",  
  "veiculoId": "456",  
  "claimType": "colisao",  
  "damageDescription": "Batida traseira"  
}
```


CORS

Como o frontend e o backend estão no **mesmo domínio** (Next.js API Routes), não há problema de CORS. O browser não bloqueia pedidos same-origin. Num cenário de domínios separados, seria necessário configurar headers **Access-Control-Allow-Origin**.

9. Mecanismos de Segurança Implementados

Mecanismo	Onde	Como Funciona
Verificação de JWT	lib/auth-server.ts	Todo o pedido verifica a assinatura criptográfica do token Firebase antes de executar qualquer lógica
Separação de credenciais	.env + Next.js	FIREBASE_PRIVATE_KEY nunca chega ao browser. Prefixo NEXT_PUBLIC_ é obrigatório para variáveis expostas ao cliente
Admin SDK server-only	lib/firebase-admin.ts	O Firestore é acedido exclusivamente pelo servidor com credenciais de serviço, nunca pelo cliente
Validação de entrada	app/api/*/route.ts	Campos obrigatórios verificados antes de gravar. Pedidos inválidos retornam 400 Bad Request
Tratamento de erros	Todos os route handlers	Try/catch em todas as operações Firestore, erros não expõem stack traces ao cliente
CORS implícito	Next.js	Same-origin por padrão, sem exposição de headers desnecessários
Secrets Manager	apphosting.yaml	Chaves de produção geridas pelo Firebase Secrets Manager, não no código-fonte
TypeScript strict	tsconfig.json	strict: true previne null/undefined não tratados que poderiam causar crashes

Nota: O sistema não implementa RBAC (controlo de acesso baseado em papéis) granular por endpoint — qualquer utilizador autenticado acede a todos os recursos. Para uma versão de produção completa, seria necessário verificar o campo tipoAcesso do utilizador em cada rota. Este ponto é identificado como dívida técnica.

10. Tratamento de Formulários Inválidos

No Frontend — React Hook Form + Zod

A validação acontece **antes** de qualquer pedido HTTP:

- 1. O utilizador tenta submeter um formulário vazio
- 2. O React Hook Form executa o schema Zod
- 3. Erros são mapeados para cada campo individualmente

4. A UI mostra mensagens inline ("Campo obrigatório", "E-mail inválido")
5. O pedido à API **nunca é feito** — o `fetch()` nem chega a ser chamado

Exemplo visual:

```
[ Nome: _____ ] ← "Campo obrigatório" (vermelho)
[ E-mail: _____ ] ← "Formato inválido" (vermelho)
[ Botão SALVAR ]      ← desativado ou bloqueado
```

No Backend — Route Handlers

Se um dado inválido escapar ao frontend (ex: via Postman ou ataque direto):

1. O `requireAuth()` verifica o token → 401 se ausente
2. O handler verifica campos obrigatórios → 400 com mensagem descritiva
3. O Firestore rejeita tipos incompatíveis → erro capturado pelo try/catch
4. O handler retorna `{ error: "Mensagem descritiva" }` com status HTTP adequado
5. O frontend exibe um toast de erro ao utilizador

O sistema tem dupla camada de validação: client-side para UX rápida, server-side para segurança real.

11. Simulação de Banca — 5 Perguntas Difíceis

Pergunta 1 — Desnormalização no Firestore

"No vosso schema, os dados do cliente, veículo e oficina estão duplicados dentro do documento do sinistro como 'snapshots'. Isto viola a Primeira Forma Normal do modelo relacional. Como justificam esta decisão e quais são os riscos?"

Resposta: A decisão é intencional e baseada nas características do Firestore (NoSQL). O Firestore não suporta JOINS — para mostrar o nome do cliente num sinistro, seria necessário um segundo pedido à base de dados. Com snapshots desnormalizados, uma única leitura do documento `sinistro` contém todos os dados necessários para exibir o card no Kanban.

O risco real é a **inconsistência eventual**: se o telefone do cliente for atualizado, os sinistros antigos mantêm o telefone antigo no snapshot. Esta é uma troca deliberada: no contexto de sinistros, os dados históricos devem refletir o estado no momento do sinistro, não o estado atual. É o mesmo padrão usado em faturas — uma fatura não muda o preço se o produto for alterado depois.

Pergunta 2 — Escalabilidade da Classificação em Memória

"A API do dashboard carrega todos os sinistros em memória para classificá-los nas 5 colunas do pipeline. O que acontece quando a base de dados tiver 100.000 sinistros? Isto é escalável?"

Resposta: Não é escalável para volumes muito grandes — é uma limitação reconhecida da implementação atual. A solução correta seria armazenar o `pipelineStatus` como campo indexado em cada sinistro e fazer queries filtradas diretamente no Firestore. Optámos pela abordagem em memória porque: (1) o Firestore

limita queries compostas e exige índices manuais para cada combinação de filtros; (2) o volume de sinistros activos num sistema de perícias é tipicamente baixo (dezenas a centenas, não milhões); (3) simplifica significativamente a lógica de negócio.

Para escalar, o próximo passo seria usar **Cloud Functions** para manter o campo **kanbanColumn** atualizado a cada mudança de estado, transformando a classificação numa query simples indexada.

Pergunta 3 — Controlo de Acesso por Perfil

"Qualquer utilizador autenticado consegue ler e apagar sinistros de outros utilizadores? Como o sistema garante que um vistoriador não apaga um sinistro que não lhe pertence?"

Resposta: Essa é uma vulnerabilidade real no estado atual. O **requireAuth()** verifica apenas que o utilizador está autenticado, mas não verifica se tem permissão para aceder ao recurso específico. Um vistoriador autenticado poderia, via API direta, apagar um sinistro de outro utilizador.

A mitigação incompleta existente é que a interface web não expõe esse botão a vistoriadores. Mas security-by-obscurity não é segurança real. A solução correta seria verificar o campo **tipoAcesso** do utilizador em cada endpoint e implementar regras no Firestore Security Rules como camada adicional. Identificamos isto como dívida técnica para a versão 1.0.

Pergunta 4 — Firestore vs PostgreSQL

"Por que escolheram Firestore (NoSQL) em vez de PostgreSQL para um sistema com dados claramente relacionais (sinistro → cliente → veículo → vistoria)? Não seria mais natural um banco relacional?"

Resposta: É uma pergunta legítima. A escolha foi pragmática, não puramente técnica. PostgreSQL seria mais natural para as relações — as queries de "todos os sinistros do cliente X com vistoria pendente" seriam um JOIN simples. Escolhemos Firestore pelos seguintes motivos: (1) integração nativa com Firebase Auth — o mesmo SDK; (2) deploy sem gestão de servidor (serverless); (3) escalabilidade automática para picos de carga; (4) custo zero no tier gratuito para um protótipo/TCC.

A contrapartida foi paga em complexidade de queries — como vemos nas classificações em memória e nos snapshots desnormalizados. Se o sistema fosse para produção com volume real, PostgreSQL com Supabase seria a nossa recomendação.

Pergunta 5 — Segurança do Token JWT

"O token Firebase JWT dura 1 hora. O que acontece se um token for roubado durante essa janela? Existe alguma forma de invalidar um token antes de expirar?"

Resposta: Excelente pergunta sobre um limite fundamental do JWT. Por design, tokens JWT são stateless — uma vez emitidos, o servidor não tem forma de invalidá-los individualmente antes da expiração, pois a validade está codificada no próprio token.

O Firebase Admin SDK oferece uma solução parcial: **revokeRefreshTokens(uid)** invalida todos os refresh tokens do utilizador, impedindo a emissão de novos ID tokens. Mas os tokens já emitidos continuam válidos pelos seus 60 minutos restantes.

As mitigações que temos: (1) o token viaja apenas em HTTPS, reduzindo o risco de interceptação; (2) tokens não são armazenados em localStorage (vulnerável a XSS) — o Firebase SDK os mantém em memória; (3) a janela de 1 hora limita a exposição. Para sistemas de alta segurança, a solução seria reduzir a validade para 5–15 minutos e implementar refresh automático, o que o Firebase já faz internamente.

12. Atualização em Tempo Real e Integrações Externas

Como Funciona a Atualização nas Telas

Não usamos Firestore `onSnapshot` (push-based). O sistema usa uma abordagem **pull-based com SWR polling** — a tela pergunta ao servidor periodicamente se há dados novos.

Kanban de Orquestração — Atualiza a cada 3 segundos

```
// Configuração no kanban-board.tsx
refreshInterval: isDragging ? 0 : 3000;
```

É a tela mais "em tempo real" do sistema. A cada 3 segundos, o SWR faz um `GET /api/sinistros?tipo=kanban` silenciosamente em background. Se os dados mudaram, o React re-renderiza apenas os cards afetados. Durante um drag-and-drop, o polling é **pausado** para não causar conflito com o estado local.

Lista de Vistorias — Atualiza a cada 30 segundos

```
// Configuração no use-vistorias-list.ts
refreshInterval: 30_000;
// "Revalida a cada 30s para capturar mudanças de status no Firestore"
```

Ritmo mais lento porque vistorias mudam com menor frequência.

Dashboard — Sem polling automático

O dashboard **não faz refresh automático**. Os dados são carregados uma vez ao entrar na página via `useEffect`. Para ver dados novos, o utilizador precisa recarregar a página. Isso é uma limitação conhecida.

Após Qualquer Mutaç o — Refresh imediato

Quando o utilizador faz uma a o (vincular oficina, confirmar check-in, rejeitar vistoria), o c digo chama `mutate()` do SWR imediatamente ap s a resposta da API, sem esperar o pr ximo intervalo:

```
Utilizador clica "Confirmar Check-in"
  ↓ POST /api/sinistros/{id}/checkin
  ↓ resposta 200 OK
  ↓ mutate() → for a novo GET imediato
  ↓ Card do Kanban move de coluna na hora
```

Comparação de Abordagens

Abordagem	O que usamos	O que seria mais avançado
Mecanismo	SWR polling (pull)	Firestore <code>onSnapshot</code> (push)
Latência	0–3s (Kanban) / 0–30s (Vistorias)	Milissegundos
Carga no servidor	Vários GETs por minuto por utilizador	Uma conexão persistente
Complexidade	Simples	Requer gestão de listeners e cleanup

A escolha pelo polling foi deliberada: é mais simples de implementar, depurar e não exige gestão de conexões persistentes. Para um TCC, é uma decisão justificável.

Integrações com APIs Externas

O sistema integra apenas com o ecossistema **Firestore**. Não há chamadas a APIs externas de terceiros.

Serviço	Integrado?	Observação
Firestore Auth	Sim	Login email/senha + Google OAuth
Cloud Firestore	Sim	Base de dados principal
Firestore Admin SDK	Sim	Verificação de tokens no servidor
Firestore App Hosting	Sim	Plataforma de deploy
Serviço de e-mail externo	Não	O <code>aprovar-email</code> usa <code>generatePasswordResetLink()</code> do Firestore — o Firestore envia o e-mail, não nós
API de IA / LLM	Não	Os "alertas de IA" visíveis no sistema são dados estáticos/de demonstração. O motor de IA não está implementado
SMS / WhatsApp	Não	—
Pagamento	Não	—
API da SEFAZ / Detran	Não	—

Perguntas Difíceis Sobre Tempo Real e Integrações

Pergunta 6 — Sobre o motor de IA:

"Vocês mencionam 'alertas com inteligência artificial' no sistema. Como funciona esse motor de IA? Quais algoritmos ou APIs utilizam?"

Resposta: Os alertas exibidos no sistema representam o modelo de dados e a interface já preparados para receber alertas gerados por IA, mas o motor de geração automática ainda não foi implementado. No estado atual, os alertas são criados manualmente ou como demonstração de conceito. A arquitetura está desenhada para que um serviço externo — como uma Cloud Function conectada à API Gemini do Google — consuma os dados do Firestore, detecte padrões (SLA excedido, múltiplas rejeições de vistoria, histórico de risco do cliente) e grave alertas automaticamente na coleção **alertas**. A interface já consome e exibe esses alertas corretamente. Este é um ponto de evolução identificado para trabalhos futuros.

Pergunta 7 — Sobre escalabilidade do polling:

"O polling a cada 3 segundos no Kanban não sobrecarrega o servidor quando há muitos utilizadores simultâneos?"

Resposta: É uma preocupação válida. Com N utilizadores na tela do Kanban, o servidor recebe N requisições a cada 3 segundos. O Firebase App Hosting escala automaticamente as instâncias de servidor (Cloud Run), e o Firestore cobra por leitura, não por concorrência. O custo real seria nas leituras do Firestore: cada polling lê todos os sinistros activos. A solução de escala seria implementar **onSnapshot** — uma única conexão persistente por utilizador que recebe apenas os documentos que mudaram, em vez de todos a cada 3 segundos. Para o volume de utilizadores atual e para o contexto do TCC, o polling é aceitável.

Estas respostas cobrem o código real do sistema. Para a defesa, o mais importante é conhecer as limitações e saber justificá-las como decisões conscientes de trade-off, não como erros.